

Yêu cầu/Mục tiêu

- Transformer là một kiến trúc quan trọng, được dùng trong các LLMs hiện nay
- Trình bày hiểu biết của em về Transformer, viết bằng tiếng Việt, tất cả các em cùng viết vào file này
- Mỗi em viết nội dung của mình (đồng thời đọc nội dung của người khác, có thể thảo luận bằng cách đặt câu hỏi, comment vào nội dung của người khác)
- Quang và Hà lead nội dung này, chia task, assign task cho các bạn khác (các em có thể chọn viết phần mà mình tâm đắc hoặc thể mạnh, phần này các em tự điều phối với nhau)
- Các em viết tự do, độ dài không giới hạn, 2 bạn Quang và Hà tổng hợp lại thành tài liệu hoàn chỉnh
- Mục tiêu: sau 1 tuần, cả nhóm cùng hoàn thành 1 tài liệu để cùng hiểu sâu và rõ ràng về Transformer

If you can't explain it to a six year old, you don't understand it yourself.

Nếu bạn không thể giải thích điều gì đó cho một đứa trẻ 6 tuổi, thì bạn cũng chưa thật sự hiểu điều đó

bối cảnh → cơ chế → chi tiết kiến trúc → tác động.

PHẦN 1 — NHÓM BỐI CẢNH

[BỐI CẢNH - Sequence Transduction & RNN/LSTM] [Trần Nguyễn]

Mục tiêu

- Bản chất của bài toán Sequence Transduction
- Cơ chế hoạt động cơ bản của Seq2Seq
- Nguyên nhân các kiến trúc Recurrent Neural Networks (RNN/LSTM) truyền thống lại gặp rào cản lớn về tốc độ và khả năng lưu trữ thông tin dài hạn.

Bối cảnh

- Trước đây, Deep Neural Networks(DNN) rất mạnh mẽ trong việc xử lý các vấn đề khó khăn như nhận dạng đối tượng trực quan, Tuy nhiên, khi áp dụng DNN cho bài toán Sequence Transduction - bài toán ánh xạ 1 chuỗi đầu vào sang 1 chuỗi đầu ra, điển hình là dịch máy hay nhận dạng giọng nói, DNN lại tỏ ra kém hiệu quả. Lí do là vì DNN bị giới hạn bởi việc đầu vào và ra phải có 1 số chiều nhất định, mà các bài toán như dịch

máy lại là các bài toán chuỗi có độ dài đầu vào và đầu ra không xác định trước, đồng thời thứ tự giữa các từ lại rất phức tạp.

- Để giải quyết vấn đề này, kiến trúc Sequence to Sequence(Seq2Seq) ra đời.

Ý tưởng chính

- Kiến trúc Seq2Seq xử lý vấn đề chuỗi có độ dài linh hoạt bằng cách chia mô hình thành 2 khối chính. Khối đầu tiên, sử dụng 1 mạng RNN/LSTM đóng vai trò là Encoder để đọc toàn bộ chuỗi văn bản đầu vào từng bước một, sau đó nén toàn bộ ý nghĩa của chuỗi đó thành 1 vector duy nhất có kích thước cố định. Tiếp theo, khối thứ 2, một mạng RNN khác đóng vai trò là Decoder sẽ sử dụng chính vector cố định này để giải mã và sinh ra chuỗi đầu ra với độ dài bất kỳ.
- Bản chất của RNN/LSTM là duyệt qua dữ liệu theo từng bước thời gian, tức là tại bước t , trạng thái ẩn h_t được tính dựa trên đầu vào hiện tại và trạng thái ẩn trước đó h_{t-1} .

Hạn chế

- Vấn đề Sequential Processing: Vì h_t bắt buộc phải chờ h_{t-1} tính xong, mô hình phải tính tuần tự từ trái qua phải. Điều này sẽ khiến cho tốc độ train bị kéo dài đáng kể, gây ra giới hạn về bộ nhớ khi phải xử lý 1 vector quá dài.
- Để mô hình học được mối liên hệ từ giữa từ đầu tiên và từ cuối cùng trong câu, thông tin phải được truyền qua $O(n)$ bước. Vector đầu vào càng dài, quá trình lan truyền ngược càng dễ dẫn đến hiện tượng vanishing gradient, khiến mô hình thất bại trong việc học logic giữa các từ nằm cách xa nhau trong câu.
- Việc Encoder bắt buộc phải nén toàn bộ thông tin của 1 câu dù ngắn hay dài vào đúng 1 vector có số chiều cố định tạo ra 1 bottleneck về bộ nhớ. Càng về sau, thông tin của những từ ở đầu câu càng dễ bị mờ nhạt hoặc biến mất do mô hình phải học thêm thông tin của những từ ở cuối câu vào cùng 1 vector. Thực tế, long-term dependencies thường bị suy giảm nghiêm trọng sau khoảng 20 từ.

[Attention trước Transformer: Bahdanau (2014)] [Nhật Minh]

1. Mục tiêu

Phần này giúp người đọc hiểu:

- Vì sao mô hình Seq2Seq ban đầu hoạt động chưa tốt với câu dài
- Attention ra đời để giải quyết vấn đề gì
- Cách mô hình tập trung vào các từ quan trọng khi dịch

2. Vấn đề / Bối cảnh

Mô hình Seq2Seq ban đầu gồm 2 khối chính là encoder và decoder. encoder có vai trò đọc toàn bộ chuỗi văn bản đầu vào từng bước một, sau đó nén toàn bộ ý nghĩa của chuỗi đó thành 1 vector ngữ cảnh duy nhất có kích thước cố định. Sau đó decoder sẽ sử dụng chính vector ngữ cảnh này để sinh ra chuỗi đầu ra với độ dài bất kỳ

Vấn đề gặp phải: Khi input quá dài, việc nén toàn bộ thông tin trong input vào 1 vector ngữ cảnh với kích thước cố định gây ra sự mất mát thông tin. Câu càng dài thì thông tin càng nhiều và vector ngữ cảnh không thể nén được hết thông tin lại, từ đó giảm độ chính xác khi decoder thực hiện sinh chuỗi đầu ra

⇒ Bahdanau Attention ra đời để giải quyết vấn đề này

3. Ý tưởng chính

Thay vì nén toàn bộ thông tin của input vào 1 vector ngữ cảnh duy nhất để decoder sử dụng thì trong Bahdanau Attention, với mỗi bước dịch của decoder, ta sẽ tính lại vector ngữ cảnh để xác định xem nên tập trung vào phần nào của input hay nói cách khác, để tính độ tương quan giữa các từ ở input với từ đang xét ở output

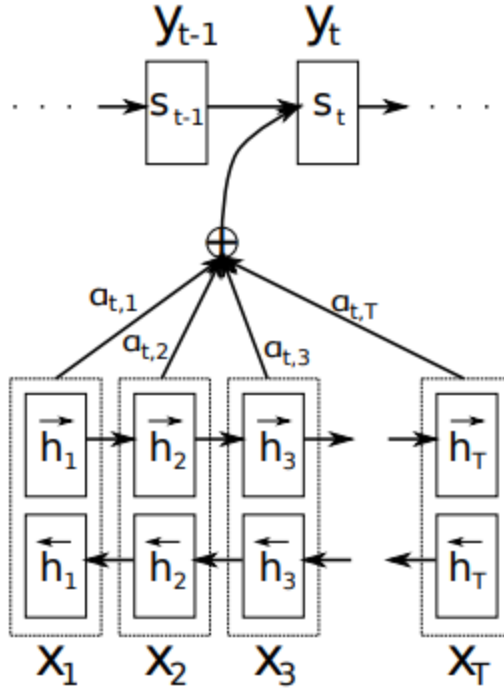
4. Cơ chế hoạt động

Encoder:

Ở phần này cấu trúc được sử dụng là bidirectional RNN để tổng hợp thông tin không chỉ của từ đang xét mà của cả các từ xung quanh nó. BiRNN gồm 2 phần:

- Phần RNN tiến xét các từ theo thứ tự từ x_1 đến x_T với T là độ dài của input, sau đó tính các hidden state tiến tương ứng là $h_1, h_2, \dots, h_T$ (tiền). Các hidden state này chứa thông tin từ đầu câu đến từ hiện tại
- Phần RNN lùi xét các từ theo thứ tự ngược lại, từ x_T đến x_1 , sau đó tính các hidden state lùi tương ứng là $h_T, h_{T-1}, \dots, h_1$ (lùi). Các hidden state này chứa thông tin từ cuối câu đến từ hiện tại

Khi này tại mỗi vị trí i , BiRNN tạo ra 2 hidden state là h_i (tiền) và h_i (lùi), ta tiến hành ghép 2 vector hidden state này lại thành 1 vector hidden state tổng hợp là $h_i = [h_i \text{ (tiền)}; h_i \text{ (lùi)}]$. Các hidden state này sẽ được chuyển cho decoder sử dụng



Decoder:

Tại mỗi bước thứ i , xác suất để decoder sinh ra từ y_i khi biết các từ đã sinh ra trước đó cùng

với input là: $p(y_i | y_1, \dots, y_{i-1}, \mathbf{x}) = g(y_{i-1}, s_i, c_i)$

trong đó g là 1 hàm phi tuyến, s_i là hidden state của decoder tại bước thứ i :

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

với y_{i-1} là từ được sinh ra ở bước thứ $i - 1$, s_{i-1} là hidden state tại bước thứ $i - 1$, c_i là vector ngữ cảnh được tính tại bước thứ i còn f là 1 hàm phi tuyến. Như đã nói ở trên, vector ngữ cảnh c_i này được tính riêng cho mỗi bước i bằng cách tính tổng có trọng số của các hidden state h_j đã tính ở phần encoder:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

Các trọng số $\alpha_{i,j}$ thể hiện mức độ quan trọng của hidden state h_j với decoder tại bước sinh output thứ i , thực chất là các alignment score $e_{i,j}$ dùng để đo sự liên quan giữa hidden state thứ $i - 1$ ở output với phần input xung quanh vị trí j , sau khi thông qua hàm softmax để chuẩn hóa về khoảng $[0, 1]$. Các tham số $e_{i,j}$ được tính như sau:

$$e_{ij} = a(s_{i-1}, h_j)$$

trong đó a là một mạng nơ-ron truyền thẳng (FFN)

5. Kết luận

- Bahdanau Attention cho decoder nhìn lại toàn bộ input để tính vector ngữ cảnh
- Vector ngữ cảnh thay đổi theo từng bước decode
- Vẫn còn hạn chế là xử lý tuần tự nên huấn luyện chậm và chưa giải quyết được vanishing gradient
- Đây là nền móng của cơ chế Attention trong Transformer

6. Tham khảo

<https://arxiv.org/pdf/1409.0473>

PHẦN 2 —KIẾN TRÚC TRANSFORMER

Tổng quan Encoder-Decoder Architecture: [HLong]

1. Tổng quan Block Diagram: N=6 layers Encoder + N=6 layers Decoder

Mô hình gốc (được giới thiệu trong bài báo *Attention Is All You Need*) sử dụng một ngăn xếp (stack) gồm 6 layer cho Encoder và 6 layer cho Decoder.

- **Encoder Stack (N=6):** Các layer này hoàn toàn giống nhau về cấu trúc nhưng không chia sẻ trọng số (weights). Nhiệm vụ của khối Encoder là đọc toàn bộ chuỗi đầu vào cùng một lúc và trích xuất ra một biểu diễn ngữ cảnh cực kỳ sâu sắc cho từng từ (token) dựa trên mối quan hệ của nó với tất cả các từ khác trong câu.
- **Decoder Stack (N=6):** Tương tự, Decoder cũng có 6 layer. Nhiệm vụ của nó là sinh ra chuỗi đầu ra theo từng bước (autoregressive), dựa vào ngữ cảnh đã được Encoder trích xuất và các từ đã được sinh ra trước đó.

2. Cấu trúc chi tiết và Tác dụng của mỗi Layer

Mỗi layer trong Encoder và Decoder đều được xây dựng từ các module tiêu chuẩn. Điểm khác biệt duy nhất là Decoder có thêm một khối Cross-Attention ở giữa.

A. Multi-Head Attention (MHA)

- Tác dụng: Cho phép mô hình tập trung (attend) vào các phần khác nhau của chuỗi đầu vào cùng một lúc.
- Trong Encoder: Là *Self-Attention*. Query (Q), Key (K), và Value (V) đều đến từ cùng một nguồn (đầu ra của layer trước).
- Trong Decoder:
 - *Masked Self-Attention*: Ngăn không cho Decoder "nhìn trộm" các từ ở tương lai trong quá trình huấn luyện. Q, K, V đến từ đầu ra của Decoder layer trước.
 - *Cross-Attention (Encoder-Decoder Attention)*: Q đến từ layer trước của Decoder, trong khi K và V đến từ đầu ra cuối cùng của Encoder. Điều này giúp Decoder tập trung vào các phần thông tin liên quan từ chuỗi gốc khi dịch hoặc sinh văn bản.

B. Add & Norm (Residual Connection & Layer Normalization)

- Tác dụng: Nằm ngay sau mỗi sub-layer (MHA và FFN).
- Add (Residual Connection): Cộng thẳng đầu vào của sub-layer với đầu ra của nó: $\text{Output} = x + \text{Sublayer}(x)$. Điều này giúp giải quyết bài toán triệt tiêu đạo hàm (vanishing gradient), cho phép huấn luyện các mạng rất sâu (như 6 layers hay thậm chí 96 layers).
- Norm (Layer Normalization): Chuẩn hóa kết quả trên từng token độc lập, giúp mô hình hội tụ nhanh hơn và quá trình huấn luyện ổn định hơn.

C. Feed-Forward Network (FFN)

- Tác dụng: Là một mạng nơ-ron truyền thẳng gồm 2 lớp tuyến tính (Linear) xen kẽ bằng một hàm kích hoạt (thường là ReLU hoặc GELU). Trong khi Attention giúp các từ "giao tiếp" với nhau, FFN hoạt động độc lập trên từng vị trí từ (position-wise) để chuyển đổi và tăng cường khả năng biểu diễn phi tuyến tính của mô hình.

3. Phân tích các Siêu tham số (Hyperparameters)

Mô hình Transformer chuẩn sử dụng các thông số không gian (dimensions) sau:

- $d_{\text{model}} = 512$: Kích thước của vector đại diện cho mỗi token xuyên suốt toàn bộ mô hình (từ Embedding cho đến đầu ra của mỗi layer). Việc giữ nguyên kích thước này là bắt buộc để có thể thực hiện phép cộng Residual Connection (Add).
- $h = 8$: Số lượng "cái đầu" (heads) trong Multi-Head Attention. Thay vì tính Attention một lần với vector 512 chiều, mô hình chia nhỏ nó ra làm 8 phần để học các khía cạnh ngữ nghĩa khác nhau song song.
- $d_k = d_v = 64$: Kích thước của ma trận Key (K) và Value (V). Nó được tính tự động bằng cách chia đều không gian model cho số lượng head: $d_k = d_{\text{model}} / h = 512 / 8 = 64$.
- $d_{\text{ff}} = 2048$: Kích thước lớp ẩn (hidden layer) bên trong FFN. Thông thường, mô hình sẽ phóng to vector từ 512 lên 2048 chiều (để tạo không gian trích xuất đặc trưng rộng hơn), sau đó lại nén về 512 chiều trước khi đi qua Add & Norm.

4. Luồng dữ liệu (Data Flow) tổng thể

Đây là hành trình của dữ liệu từ lúc đi vào cho đến khi dự đoán ra kết quả cuối cùng:

1. Input Embedding & Positional Encoding:

- Văn bản đầu vào được chuyển thành các token, sau đó map thành các vector số thực có kích thước $d_{\text{model}} = 512$.
- Vì Transformer không xử lý tuần tự, ta phải cộng thêm *Positional Encoding* (tín hiệu vị trí) vào các vector này để mô hình biết từ nào đứng trước, từ nào đứng sau.

2. Chạy qua Encoder Stack:

- Chuỗi vector (đã có thông tin vị trí) đi vào Layer 1 của Encoder.
- Đi qua *Self-Attention* $\rightarrow$ *Add & Norm* $\rightarrow$ *FFN* $\rightarrow$ *Add & Norm*.
- Quá trình này lặp lại y hệt qua 6 layers. Đầu ra cuối cùng là một ma trận gồm các vector 512-chiều chứa ngữ cảnh cực kỳ phong phú của câu gốc.

3. Chạy qua Decoder Stack:

- Đầu vào của Decoder (chính là chuỗi target đã được dịch sang phải - shifted right) đi qua Embedding và Positional Encoding.
- Đi vào *Masked Self-Attention* (để tránh nhìn thấy tương lai) $\rightarrow$ *Add & Norm*.
- Đi vào *Cross-Attention*: Lúc này, Decoder lấy thông tin từ câu gốc bằng cách dùng trạng thái hiện tại làm Query, và lấy ma trận Memory từ khối Encoder làm Key và Value $\rightarrow$ *Add & Norm*.
- Đi qua *FFN* $\rightarrow$ *Add & Norm*.
- Lặp lại qua 6 layers Decoder.

4. Linear $\rightarrow$ Softmax (Output Generation):

- Đầu ra của Decoder layer thứ 6 vẫn là các vector có kích thước 512.
- Khối **Linear** (Linear Classifier) sẽ phóng to vector 512 chiều này lên thành một vector có kích thước bằng với toàn bộ từ vựng (Vocabulary Size, ví dụ: 30,000 chiều).
- Hàm **Softmax** biến vector khổng lồ này thành một phân phối xác suất (tổng các giá trị = 1). Từ nào có xác suất cao nhất sẽ được chọn làm từ tiếp theo được sinh ra. Quá trình lặp lại cho đến khi sinh ra token kết thúc.

Decoder Layer chi tiết (Người viết Đức Dững)

1. Masked Multi-Head Attention

1. Vấn đề là gì

Ở đây thuật ngữ Multi-Head Attention đã được trình bày cơ chế chi tiết ở lớp Encoder trước đó bây giờ chúng ta đã có thêm 1 từ "Masked" vậy tại sao ta phải "Masked" ?

Hãy nhớ lại nguyên lý cốt lõi của khối Decoder: **Sinh văn bản tự hồi quy (Auto-regressive)**. Nghĩa là nó phải đoán từ thứ t dựa trên các từ từ 1 đến $t-1$.

Chẳng hạn như bạn đang làm một bài thi điền từ vào chỗ trống: "Tôi yêu ____".

- Để điền được chữ "mèo", bạn chỉ được phép đọc "Tôi yêu".
- Nếu hệ thống cho phép từ "yêu" nhìn thấy chữ "mèo" ở tương lai trước khi nó thực sự được sinh ra, thì đó gọi là "nhìn trộm" (data leakage/cheating). Mô hình sẽ học thuộc lòng thay vì thực sự hiểu cách dự đoán.

2. Bản chất Toán học: Sự thanh lịch của vô cực

Thuật toán Masked Self-Attention vẫn sử dụng chung nền tảng Scaled Dot-Product Attention, nhưng nó can thiệp vào ngay trước bước tính xác suất (Softmax).

Công thức gốc là:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Đặt biểu thức trong hàm softmax là 1 ma trận S

Giả sử chuỗi đầu vào Decoder của chúng ta là 3 mốc thời gian: t_1, t_2, t_3 .

Ma trận S (kích thước 3×3) ban đầu sẽ trông như thế này, trong đó s_{ij} là điểm số từ i "chú ý" vào từ j :

$$S = \begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{21} & s_{22} & s_{23} \\ s_{31} & s_{32} & s_{33} \end{bmatrix}$$

Vấn đề: Ở hàng 1 (đang xử lý t_1), nó đang có điểm số liên kết với t_2 (s_{12}) và t_3 (s_{13}). Đây là thông tin tương lai, chúng ta phải xóa nó đi!

Giải pháp: Cộng thêm Ma trận Mặt nạ (Mask Matrix - M)

Tác giả định nghĩa một ma trận M có cùng kích thước, trong đó nửa tam giác trên (tương lai) mang giá trị âm vô cực ($-\infty$), và nửa tam giác dưới (quá khứ + hiện tại) mang giá trị 0:

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Khi thực hiện phép cộng $S + M$, ta được:

$$S + M = \begin{bmatrix} s_{11} & -\infty & -\infty \\ s_{21} & s_{22} & -\infty \\ s_{31} & s_{32} & s_{33} \end{bmatrix}$$

Sức mạnh của hàm Softmax:

Hàm Softmax tính xác suất bằng cách lấy hàm mũ cơ số e của các phần tử: e^x . Theo giới hạn toán học:

$$\lim_{x \rightarrow -\infty} e^x = 0$$

Do đó, khi đưa ma trận ($S + M$) qua hàm Softmax, tất cả các ô $-\infty$ sẽ lập tức biến thành số 0 tuyệt đối. Nửa tam giác dưới sẽ được chuẩn hóa lại để tổng mỗi hàng bằng 1.

$$\text{softmax}(S + M) = \begin{bmatrix} 1.0 & 0 & 0 \\ 0.4 & 0.6 & 0 \\ 0.2 & 0.5 & 0.3 \end{bmatrix}$$

Kết quả:

- Từ t_1 (Hàng 1) bây giờ dồn 100% sự chú ý vào chính nó.
- Từ t_2 (Hàng 2) chỉ có thể phân bổ sự chú ý cho t_1 và chính nó (t_2).
- Từ tương lai hoàn toàn bị đóng băng!

=> Áp dụng vào Sơ đồ Khối Decoder

Bây giờ hãy nhìn lại bức hình tổng thể, đặc biệt là lớp "Masked Multi-Head Attention" ở nửa dưới khối bên phải:

- **Đầu vào:** Lớp này nhận luồng dữ liệu từ "Outputs (shifted right)". Đây chính là những từ mà hệ thống đã dịch được cho đến thời điểm hiện tại.
- **Bên trong lớp:** Nó thực hiện thao tác chia 8 đầu (Multi-Head) và nhân ma trận với âm vô cực ($-\infty$) y như chúng ta vừa chứng minh bằng toán học ở trên để loại bỏ thông tin tương lai.
- **Đầu ra:** Nó sinh ra một ma trận biểu diễn ngữ cảnh mang tính nhân quả (causality). Ma trận này chứa sự thấu hiểu về "những gì đã dịch" một cách hợp lệ.
- **Chuyển giao:** Ma trận này lập tức được đẩy thẳng lên lớp con tiếp theo (Multi-Head Attention ở giữa) để đóng vai trò làm Query (Q).

Chính thao tác toán học này đảm bảo quá trình dự đoán song song trong lúc huấn luyện (training) diễn ra hoàn hảo mà không làm hỏng tính chất dự đoán tuần tự lúc mô hình chạy thực tế (inference).

3. Cross Attention

Như vậy ở trên ta đã hiểu rõ về cơ chế Masked Multi Head Attention, ngay tiếp đó ở trong khối decoder xuất hiện 1 đại lượng nữa gọi là Cross Attention

Công thức cốt lõi của Cross Attention không khác gì công thức attention ở trên

Nếu quan sát kỹ khối "Multi-Head Attention" ở giữa Decoder, bạn sẽ thấy nó nhận 3 mũi tên đi vào. Ba mũi tên này không xuất phát từ cùng một nơi:

- **Mũi tên đi từ dưới lên (Tạo ra Query - \$Q\$):** Luồng dữ liệu này đi lên từ lớp Masked Multi-Head Attention của Decoder. Nó mang theo thông tin của **chuỗi ngôn ngữ đích đang được sinh ra** (những từ đã được dịch tính đến thời điểm hiện tại).
- **Hai mũi tên vắt ngang từ trái sang (Tạo ra Key - \$K\$ và Value - \$V\$):** Luồng dữ liệu này xuất phát từ tầng trên cùng của khối Encoder bên trái. Nó chứa đựng toàn bộ sự thấu hiểu về **chuỗi ngôn ngữ gốc**. Một nhánh biến thành các thẻ định danh (\$K\$), nhánh còn lại mang nội dung thực sự (\$V\$).

Sự thiết kế dây chuyền này cho phép mọi vị trí trong bộ giải mã (Decoder) có thể đối chiếu và trích xuất thông tin từ mọi vị trí trong toàn bộ chuỗi đầu vào (Encoder).

Ví dụ thực tế: Cách Cross-Attention chạy

Giả sử chúng ta đang huấn luyện mô hình dịch câu tiếng Anh "I love cats" sang câu tiếng Việt "Tôi yêu mèo".

Giai đoạn chuẩn bị (Encoder đã làm xong việc): Khối Encoder bên trái đã đọc xong câu "I love cats". Nó nén toàn bộ ngữ pháp, ngữ nghĩa của 3 từ này thành một ma trận biểu diễn khổng lồ. Ma trận này đứng đợi sẵn ở đầu ra của Encoder, sẵn sàng đóng vai trò làm K (Key) và V (Value) cho bất cứ lúc nào Decoder cần gọi đến.

Giai đoạn thực thi tại Cross-Attention (Khi Decoder đang dự đoán chữ "mèo"): Giả sử Decoder đã chạy được 2 bước và sinh ra được chuỗi [`<START>`, "Tôi", "yêu"]. Bây giờ nó cần dự đoán từ tiếp theo.

- **Bước 1: Hình thành "Nhu cầu" (Query - Q)** Lớp Masked Attention ở dưới đáy tổng hợp ngữ cảnh của chuỗi [`<START>`, "Tôi", "yêu"] thành một vector Q. Vector này mang thông điệp ngầm hiểu: "Tôi đã có chủ ngữ là người nói, có động từ chỉ cảm xúc. Bây giờ, tôi cần tìm một danh từ đóng vai trò tân ngữ để hoàn thiện ý nghĩa cho hành động 'yêu'."
- **Bước 2: Quá trình "So khớp" ($Q \times K^T$)** Vector Q này được gửi lên khối Cross-Attention. Tại đây, nó được nhân vô hướng với tất cả các vector định danh (K) đang đợi sẵn từ Encoder: K_I , K_{love} , và K_{cats} . Phép nhân này đo lường mức độ tương đồng. Vì Q đang tìm kiếm "tân ngữ", điểm số tương đồng giữa Q và K_{cats} sẽ bùng nổ rất cao, trong khi điểm số với K_I hay K_{love} sẽ rất thấp.
- **Bước 3: Chấm điểm và Trích xuất (Softmax và $\times V$)** Hệ thống đưa các điểm số này qua hàm Softmax để chuyển thành phần trăm. Từ "cats" có thể nhận được 95% sự chú ý, trong khi "I" và "love" chia nhau 5% còn lại. Cuối cùng, hệ thống lấy phần trăm này nhân với nội dung thực sự (V) của câu gốc. Kết quả là 95% hàm lượng thông tin của chữ "cats" được hút sang khối Decoder.

- **Bước 4: Sinh từ mới** Vector thông tin mang đậm bóng dáng của "cats" này tiếp tục đi lên qua lớp Feed Forward, và cuối cùng giúp hệ thống tự tin xuất ra dự đoán từ tiếng Việt tiếp theo chính là "mèo".

Tóm lại: Khối Cross-Attention chính là "bộ phận phiên dịch" thực sự trong sơ đồ này. Nếu không có nó (như trong các kiến trúc GPT), mô hình chỉ có thể tự nói chuyện với chính nó (tự hồi quy) chứ không thể tham chiếu sang một hệ thống tri thức hay một ngôn ngữ khác được.

Scaled Dot-Product Attention: [Trâm]

1. Mục tiêu

Phần này giúp người đọc hiểu được **cơ chế cốt lõi** mà Transformer dùng để biết "Token nào cần chú ý đến Token nào" khi xử lý một chuỗi. Cụ thể: $\sqrt{d_k}$; và sự khác nhau giữa scaled dot-product với các biến thể attention trước đó.

2. Bối cảnh

Trước phần này, paper đã trình bày kiến trúc tổng quát của Transformer (encoder-decoder). Vấn đề cần giải quyết: làm sao để mỗi vị trí trong chuỗi tổng hợp thông tin từ các vị trí khác một cách có chọn lọc — không phải dùng RNN (tuần tự, chậm) hay CNN (chỉ nhìn được cửa sổ cục bộ), mà bằng một phép tính song song thuần túy.

=> Attention đã tồn tại từ trước (Bahdanau 2015), nhưng Transformer loại bỏ hoàn toàn RNN và dùng attention làm cơ chế duy nhất để mô hình hóa sự phụ thuộc giữa các token - đây là điểm đột phá.

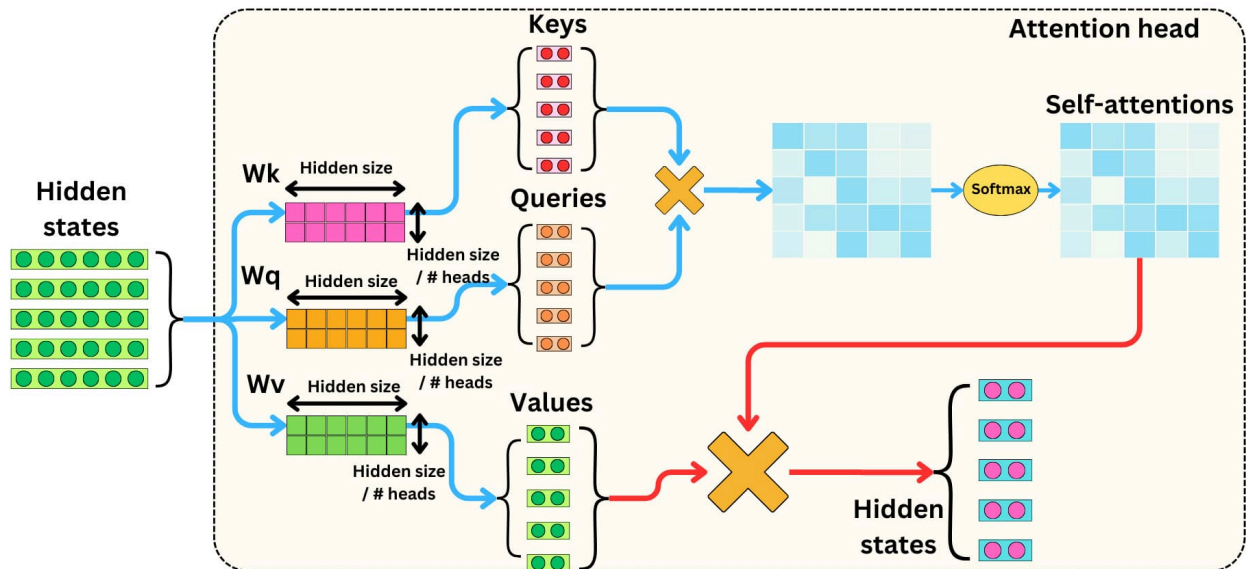
3. Ý tưởng chính

Hãy tưởng tượng mỗi token là một nhà nghiên cứu bước vào thư viện.

- **Query (Câu hỏi):** Nhu cầu tìm kiếm hiện tại của nhà nghiên cứu (token đó cần biết gì để hiểu rõ ngữ cảnh của chính mình).
- **Key (Tiêu đề):** Nhấn tóm tắt nội dung của từng cuốn sách trong thư viện (đại diện cho thông tin định danh của các token khác).
- **Value (Nội dung):** Tri thức thực sự nằm bên trong mỗi cuốn sách.

Thay vì chỉ chọn bừa hoặc đọc duy nhất một cuốn, nhà nghiên cứu sẽ **đối chiếu câu hỏi (Query) với tất cả các tiêu đề (Key)** để chấm điểm mức độ ăn khớp. Sách nào càng liên quan thì càng được ưu tiên trích xuất nội dung (**Value**).

4. Cơ chế hoạt động



① Chiều input → Q, K, V

Ma trận đầu vào X (shape: $\text{seq_len} \times d_{\text{model}}$) được nhân với ba ma trận trọng số

W_Q, W_K, W_V để tạo ra Q, K, V — mỗi cái có shape $\text{seq_len} \times d_k$.

② Tính attention scores

Nhân $Q \cdot K^T \rightarrow$ ma trận điểm số ($\text{seq_len} \times \text{seq_len}$). Ô $[i][j]$ = mức độ token i "quan tâm" đến token j .

③ Scale bởi $\sqrt{d_k}$

Chia toàn bộ điểm số cho $\sqrt{d_k}$ để tránh giá trị quá lớn, giữ gradient ổn định khi backprop qua softmax.

④ Softmax → attention weights

Áp dụng softmax theo chiều hàng → mỗi hàng là một phân phối xác suất (tổng = 1). Đây là "tỉ lệ đọc" của từng token.

⑤ Nhân với V → output

Weights $\cdot V \rightarrow$ vector context cho mỗi token, là tổng có trọng số của các Value. Output shape = $\text{seq_len} \times d_v$.

5. Công thức

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q : Query matrix: - vị trí hiện tại đang hỏi gì ($\text{seq} \times d_k$)

K : Key matrix - mỗi vị trí có thông tin gì ($\text{seq} \times d_k$)

V : Value matrix - nội dung thực tế để tổng hợp ($\text{seq} \times d_v$)

d_k : số chiều vector của các vector thuộc ma trận Q và K

Softmax: hàm chuẩn hóa được áp dụng theo từng hàng, chuyển đổi các mức điểm số thành 1 phân phối xác suất (tổng bằng 1) cho mỗi query

6. Vì sao quan trọng

Nếu loại bỏ cơ chế này, mô hình Transformer sẽ bị ảnh hưởng nghiêm trọng theo 3 khía cạnh sau:

Thành phần thiếu hụt	Nguyên nhân kỹ thuật	Hệ lụy lên mô hình
1. Hệ số Scaling $\sqrt{d_k}$	Khi d_k lớn, Softmax bị rơi vào vùng bão hòa	Triệt tiêu gradient ; mô hình không thể học ngay từ đầu <i>training</i> .
2. Phân tách Q/K/V	Không phân định được "cái tôi tìm" Q và "cái tôi có" K .	Mất tính linh hoạt ; khó học được các <i>pattern</i> phức tạp.
3. Cơ chế song song hóa	Phải tính tuần tự từng cặp (i, j) qua mạng FFN (tốn $O(n^2)$ lượt gọi).	Đánh mất sức mạnh của GPU ; không tận dụng được phép nhân ma trận $O(n^2 \cdot d)$.

7. Dễ nhầm ở đâu

1. Q, K, V là ba đầu vào (input) độc lập khác nhau

- Trong cơ chế Self-Attention, cả ba ma trận Q , K , V đều được sinh ra từ cùng một ma trận đầu vào X . Chúng chỉ là kết quả của ba phép chiếu tuyến tính khác nhau qua các ma trận trọng số tương ứng W_Q , W_K , W_V .

2. Hàm Softmax được áp dụng cào bằng cho toàn bộ ma trận

- Softmax được áp dụng độc lập theo từng hàng (row-wise). Điều này đảm bảo rằng mỗi một query token sẽ tự tạo ra một phân phối xác suất chú ý riêng biệt cho chính nó với tổng các trọng số luôn bằng 1.

MULTI-HEAD ATTENTION: [NHẬT ANH]

1. Mục tiêu

Phần này giúp người đọc hiểu **vì sao Transformer không dùng một cơ chế attention duy nhất, mà chạy nhiều cơ chế attention song song** — gọi là các **đầu (head)**. Sau khi đọc xong, bạn sẽ:

- Hiểu trực giác đằng sau ý tưởng “nhiều đầu” và tại sao một đầu là không đủ.
- Nắm được cơ chế toán học: chiếu (project) Q, K, V thành h không gian con, chạy attention độc lập, rồi gộp lại.
- Biết cách các tham số d_{model} , h, d_k , d_v liên hệ với nhau.
- Phân biệt được Multi-Head Attention với Scaled Dot-Product Attention (phần của bạn Trâm) và biết chúng lồng vào nhau ra sao.

2. Vấn đề

Ở phần trước, bạn Trâm đã trình bày **Scaled Dot-Product Attention**: với một bộ Q, K, V, ta tính một bảng trọng số attention duy nhất rồi tổng hợp giá trị. Vấn đề là **một phép attention chỉ tạo ra MỘT cách “nhìn” mối quan hệ giữa các từ**.

Trong ngôn ngữ tự nhiên, một từ có nhiều loại quan hệ cùng lúc. Xét câu *“The animal didn’t cross the street because it was tired”*. Từ **“it”** cần kết nối với **“animal”** (quan hệ tham chiếu/ngữ nghĩa), nhưng đồng thời cũng có quan hệ cú pháp với động từ **“was”**. Một bảng attention đơn lẻ buộc phải “trung bình hóa” mọi loại quan hệ này vào một con số, làm mờ thông tin.

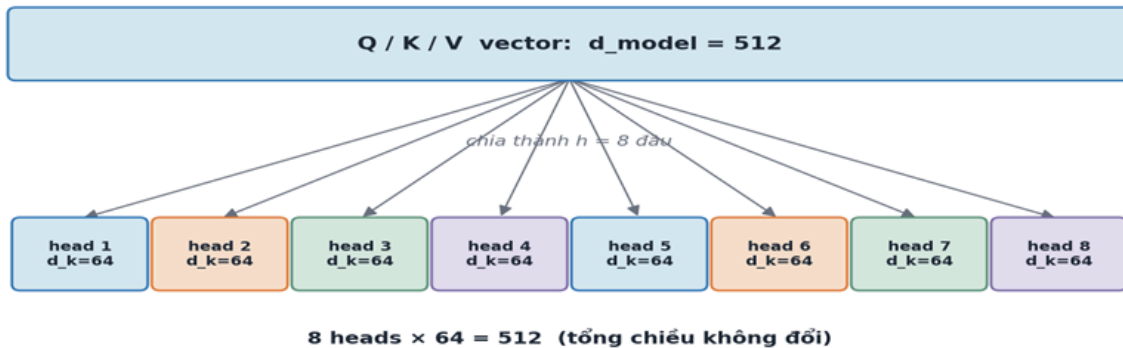
Câu hỏi đặt ra: Làm sao để mô hình học được **nhiều kiểu quan hệ khác nhau** cùng một lúc, mà vẫn giữ chi phí tính toán hợp lý?

3. Ý tưởng chính

Thay vì chạy một attention trên toàn bộ vector $d_{\text{model}} = 512$ chiều, ta **chia nhỏ thành $h = 8$ không gian con**, mỗi không gian 64 chiều. Mỗi không gian con chạy **một attention độc lập** — đó là một “đầu”. Cuối cùng ta **ghép (concat)** kết quả của 8 đầu lại và đưa qua một phép biến đổi tuyến tính.

Kiểu: hãy tưởng tượng 8 chuyên gia cùng đọc một câu. Một người chú ý đến ngữ pháp, một người chú ý đến đại từ chỉ về ai, một người chú ý đến từ lân cận... Mỗi người đưa ra góc nhìn riêng, rồi cả nhóm tổng hợp lại thành một kết luận giàu thông tin hơn bất kỳ cá nhân nào.

Chia d_{model} thành h đầu song song



4. Cơ chế hoạt động

Multi-Head Attention thực hiện 4 bước, từ input đến output:

1. **Chiếu tuyến tính (Linear projection).** Với mỗi đầu i , Q , K , V được nhân với các ma trận trọng số riêng W_i^Q , W_i^K , W_i^V để hạ chiều từ 512 xuống 64. Vì mỗi đầu có ma trận riêng nên mỗi đầu nhìn dữ liệu theo một cách khác nhau.
2. **Attention độc lập.** Mỗi đầu chạy Scaled Dot-Product Attention trên bộ (Q_i, K_i, V_i) 64 chiều của riêng nó, cho ra một vector kết quả head_i .
3. **Ghép (Concat).** 8 vector head_i (mỗi cái 64 chiều) được nối lại thành một vector $8 \times 64 = 512$ chiều.
4. **Chiếu đầu ra (W^O).** Vector ghép được nhân với ma trận W^O (512×512) để trộn thông tin giữa các đầu và đưa về đúng chiều d_{model} .

5. Công thức

Công thức tổng thể của Multi-Head Attention:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Trong đó mỗi đầu được tính bằng:

$$\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

Và Attention bên trong chính là Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

Giải thích từng ký hiệu:

Ký hiệu	Ý nghĩa	Chiều (kích thước)
Q, K, V	Query, Key, Value đầu vào	$n \times 512$
W_i^Q	Ma trận chiếu Query cho đầu i	512×64
W_i^K	Ma trận chiếu Key cho đầu i	512×64
W_i^V	Ma trận chiếu Value cho đầu i	512×64
W^O	Ma trận chiếu đầu ra (trộn các đầu)	512×512
h	Số đầu	8
$d_{\square} = d_v$	Chiều mỗi đầu = d_{model} / h	64

6. Ví dụ

Lấy câu *“The animal didn’t cross the street because it was tired”* và xét từ **“it”**. Khi xử lý **“it”**:

- **Đầu 1 (cú pháp):** học gán trọng số cao cho “animal” — giúp mô hình hiểu “it” đang chỉ con vật.
- **Đầu 2 (cục bộ):** chú ý các từ ngay cạnh như “was”, “tired” — nắm cấu trúc câu địa phương.
- **Đầu 3 (ngữ nghĩa rộng):** liên kết “it” với cả “animal” lẫn “tired” để hiểu trạng thái.

Nếu chỉ có một đầu, mô hình buộc phải chọn *một* trong các quan hệ trên hoặc trộn lẫn chúng. Nhiều đầu cho phép giữ song song mọi góc nhìn rồi gộp lại ở bước cuối.

8. Vì sao quan trọng

Nếu bỏ Multi-Head và chỉ dùng một đầu, Transformer sẽ gặp các vấn đề sau:

- Mất khả năng biểu diễn đa chiều: mô hình chỉ học được một loại quan hệ giữa các từ, làm giảm đáng kể chất lượng dịch và hiểu ngôn ngữ.
- Softmax bị “trung bình hóa”: một đầu duy nhất có xu hướng dàn trải trọng số, làm mờ các tín hiệu quan trọng.
- Bằng chứng thực nghiệm: trong bài báo gốc, giảm số đầu làm điểm BLEU giảm; quá ít đầu cho kết quả kém rõ rệt.

Multi-Head Attention cũng làm cho mô hình dễ diễn giải hơn: ta có thể trực quan hóa từng đầu để xem nó học pattern gì

9. Dễ hiểu nhầm

1. **“Nhiều đầu nghĩa là tính nhiều lần tốn kém hơn.”** Sai. Vì mỗi đầu chỉ dùng 64 chiều thay vì 512, tổng chi phí gần như tương đương một attention đầy đủ.
2. **“Các đầu được lập trình thủ công để học cú pháp/ngữ nghĩa.”** Sai. Không ai gán nhiệm vụ cho từng đầu; chúng tự học trong quá trình huấn luyện. Việc gán nhãn “đầu cú pháp” chỉ là quan sát sau khi train.
3. **“Concat rồi là xong.”** Thiếu. Sau concat bắt buộc phải qua W^O để trộn thông tin giữa các đầu; nếu bỏ W^O , các đầu không bao giờ “nói chuyện” với nhau.
4. **Nhằm d_k với d_{model} .** $d_{model} = 512$ là chiều của cả vector; $d_k = 64$ là chiều mỗi đầu. Quan hệ: $d_k = d_{model} / h$.
5. **Mỗi đầu dùng chung ma trận W .** Sai. Mỗi đầu có bộ W^q , W^k , W^v riêng — đó chính là lý do chúng học được các góc nhìn khác nhau.

10. Tóm tắt

- Multi-Head Attention chạy $h = 8$ attention song song trên các không gian con 64 chiều.
- Mỗi đầu có ma trận chiều Q/K/V riêng $\rightarrow$ học các kiểu quan hệ khác nhau (cú pháp, ngữ nghĩa, cục bộ...).
- Quy trình: Linear $\rightarrow$ Attention ($\times h$) $\rightarrow$ Concat $\rightarrow$ Linear W^O .
- Quan hệ tham số: $d_k = d_v = d_{model} / h = 512 / 8 = 64$; chi phí tổng $\approx$ một attention đầy đủ.
- Bỏ multi-head làm giảm khả năng biểu diễn và điểm BLEU; W^O là bước bắt buộc để trộn các đầu.

Positional encoding

1. Vấn đề

Bản chất của Transformer là mọi token đều được xử lý song song. Không giống RNN hay LSTM đọc tuần tự từ trái sang phải, attention nhìn toàn bộ câu cùng lúc. Vì vậy, nếu chỉ đưa embedding của từ vào mô hình, Transformer sẽ không có khái niệm nào về thứ tự.

Ví dụ, hai câu “dog bites man” và “man bites dog” gần như chứa cùng một tập token. Nếu bỏ thông tin vị trí, attention gần như không biết đâu là chủ ngữ, đâu là tân ngữ,...

2. Mục tiêu

Một positional encoding tốt phải giải quyết được nhiều yêu cầu cùng lúc:

1. **Mỗi vị trí phải có biểu diễn khác nhau**

Nếu hai vị trí cho cùng encoding thì mô hình sẽ mất thông tin thứ tự

2. **Các vị trí gần nhau nên có vector gần nhau**
 Ví dụ: pos 5 nên “gần” pos 6, pos 5 nên “xa” pos 100
 Vì trong ngôn ngữ tự nhiên, token lân cận thường liên hệ mạnh hơn token rất xa.
3. **Phải encode được khoảng cách tương đối**
 Mô hình cần học được các pattern như: “token cách 2 bước”, “token ngay trước” chứ không chỉ biết vị trí tuyệt đối.
4. **Phải tương thích với attention và đại số tuyến tính**
 Positional encoding nên có cấu trúc mà neural network dễ thao tác tuyến tính.
5. **Phải hoạt động tốt trong không gian nhiều chiều**
 Embedding thường có: 512, 768, 4096 chiều. Positional encoding cũng cần tận dụng không gian nhiều chiều này thay vì chỉ dùng một scalar đơn giản.
6. **Các vị trí phải gần như khác biệt trên context dài**
 Nếu encoding lặp quá sớm vd: pos 100 và pos 1000 có thể bị nhầm
7. **Phải smooth và liên tục**
 Nếu vector vị trí thay đổi quá đột ngột: attention khó học pattern. Một không gian liên tục giúp mô hình tổng quát hóa tốt hơn.
8. **Phải ngoại suy được cho sequence dài hơn**
9. **Không được phá hỏng ý nghĩa của embedding**

3. Công thức

Nếu ta chỉ dùng số nguyên đơn giản như:

$$PE(pos) = pos$$

thì positional encoding chỉ là một scalar thô. Nhưng embedding của token thường có hàng trăm hoặc hàng nghìn chiều. Việc thêm một con số đơn giản như vậy không cung cấp đủ cấu trúc hình học để attention khai thác. Quan trọng hơn, các số nguyên không có tính chất đại số hữu ích nào giúp Transformer học quan hệ vị trí dễ dàng.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Sin và cos được chọn vì chúng có tính chất:

$$\sin(a+b) = \sin a \cos b + \cos a \sin b$$

$$\cos(a+b) = \cos a \cos b - \sin a \sin b$$

Nhờ đó, vector tại vị trí pos+k có thể được suy ra tuyến tính từ vector tại vị trí pos.

$$\begin{bmatrix} \sin(pos + k) \\ \cos(pos + k) \end{bmatrix} = \begin{bmatrix} \cos k & \sin k \\ -\sin k & \cos k \end{bmatrix} \begin{bmatrix} \sin(pos) \\ \cos(pos) \end{bmatrix}$$

Ma trận ở giữa là ma trận quay. Điều này có nghĩa là “dịch vị trí” tương đương với một phép biến đổi tuyến tính đơn giản. Đây là điều neural network học rất tốt vì Transformer chủ yếu dùng phép nhân ma trận và linear layer.

Nếu chỉ dùng một hàm sin duy nhất thì encoding sẽ lặp chu kỳ:

$$\sin(x) = \sin(x + 2\pi)$$

nên nhiều vị trí khác nhau sẽ cho cùng giá trị. Vì vậy Transformer dùng nhiều tần số khác nhau. Các chiều tần số cao giúp phân biệt vị trí gần nhau. Các chiều tần số thấp giúp biểu diễn khoảng cách xa.

Về mặt hình học, mỗi cặp $(\sin, \cos)$ là một điểm quay trên đường tròn đơn vị. Nhiều cặp với nhiều tần số khác nhau tạo ra một quỹ đạo phức tạp trong không gian nhiều chiều. Nhờ đó, mỗi vị trí có biểu diễn gần như duy nhất nhưng vẫn liên tục và mượt mà.

Một ưu điểm khác là sinusoidal positional encoding có thể ngoại suy tốt. Vì nó là hàm toán học cố định nên mô hình vẫn tính được positional encoding cho câu dài hơn lúc train.

4. Tại sao lại là cộng chứ không phải concat

Transformer cộng positional encoding vào embedding vì cách này giữ nguyên kích thước vector và buộc semantic information lẫn positional information cùng tồn tại trong một không gian biểu diễn.

Nếu dùng concatenate:

$$x_i = [e_i; p_i]$$

thì sẽ phát sinh nhiều vấn đề.

Thứ nhất, kích thước vector tăng lên.

Ví dụ:

- embedding: 768 chiều
- positional: 768 chiều

Concat sẽ thành: 1536 chiều.

Điều này làm:

- attention matrix lớn hơn
- số tham số tăng mạnh
- chi phí tính toán tăng

Thứ hai, concatenate tách semantic và position thành hai vùng riêng biệt.

Ví dụ:

$$[e_1, e_2, \dots, p_1, p_2, \dots]$$

Attention phải tự học cách kết hợp hai phần này lại.

Ngược lại, khi cộng:

$$e_i + p_i$$

thì mỗi chiều embedding ngay lập tức mang cả:

- semantic signal
- positional signal

Điều này khiến attention khai thác position tự nhiên hơn.

Quan trọng hơn, attention chủ yếu dùng dot product:

$$QK^T$$

Nếu:

$$x_i = e_i + p_i$$

thì:

$$x_i^\top x_j = e_i^\top e_j + e_i^\top p_j + p_i^\top e_j + p_i^\top p_j$$

Ta thấy semantic và position tương tác trực tiếp trong attention score.

Nếu concatenate:

$$x_i = [e_i \quad p_i]$$

Thì:

$$\mathbf{x}_i^\top \mathbf{x}_j = \mathbf{e}_i^\top \mathbf{e}_j + \mathbf{p}_i^\top \mathbf{p}_j$$

Hai phần gần như tách biệt. Mô hình khó học quan hệ giữa token gì ở đâu

Một lý do khác là positional encoding thường có magnitude nhỏ và smooth. Khi cộng vào embedding, nó hoạt động giống một “nhiều có cấu trúc”, chỉ hơi dịch chuyển vector semantic thay vì phá hỏng nó.

5. Alternatives

Nhiều phương pháp positional encoding mới đã được đề xuất để cải thiện khả năng học quan hệ vị trí, đặc biệt trên context dài. Các phương pháp phổ biến hiện nay gồm learned positional embeddings, ALiBi và RoPE.

1. Learned Positional Embeddings (Sử dụng trong BERT, GPT-2/3)

Thay vì sử dụng các hàm lượng giác cố định, phương pháp này coi mỗi vị trí trong chuỗi đầu vào là một "token" và học các embedding vectors cho các vị trí này trong quá trình huấn luyện mạng Neural.

Cơ chế hoạt động: Mô hình duy trì một ma trận trọng số $E_{pos} \in \mathbb{R}^{L \times d}$, trong đó L là độ dài tối đa của chuỗi và d là số chiều của không gian nhúng. Vector vị trí thứ i sẽ được cộng trực tiếp vào vector ngữ nghĩa của token ở vị trí tương ứng.

Nhược điểm: Khả năng ngoại suy rất kém. Mô hình không thể xử lý các chuỗi đầu vào dài hơn L .

2.5. Encoder

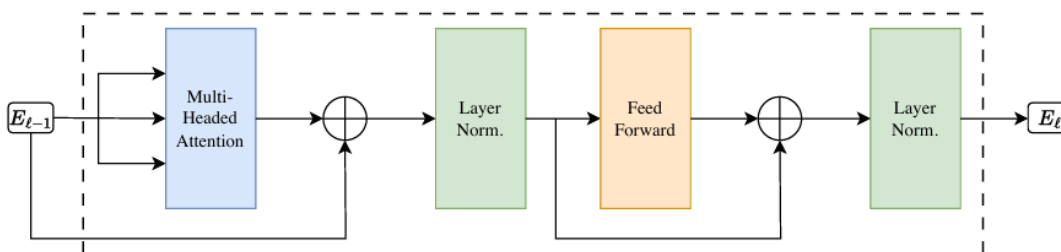


Figure 5: Encoder layer of the original Transformer architecture. Note that the $\oplus$ sign in the diagram corresponds to the skip connections.

Encoder được cấu tạo từ một chuỗi gồm N lớp encoder. Mỗi lớp encoder bao gồm các thành phần sau: Đầu tiên: Input $\rightarrow$ Tokenization $\rightarrow$ Embedding vector ($\text{len} \times 512$)

- + Cộng Positional Encoding
- + Lặp 6 lần (mỗi lần là một tầng encoder):
 - a. Multi-Head Self-Attention → cộng residual → LayerNorm
 - b. Feed-Forward Network → cộng residual → LayerNorm

Output encoder của tầng 6 → decoder

Thành phần	Mô tả	Input	Output	Tham số chính
Multi-Headed Attention	Attention đa đầu dạng self-attention: query, key, value đều từ cùng một nguồn ($E_{\{t-1\}}$). Cho phép mỗi token học quan hệ với các token khác.	(L, d_model) với d_model=512	(L, d_model)	- h=8 heads, mỗi head có W^Q, W^K, W^V kích thước (512, 64) - W^O kích thước (512, 512)
$\oplus$ (Skip connection)	Cộng đầu vào của sub-layer ($E_{\{t-1\}}$) với đầu ra của Multi-Headed Attention. Giảm hiện tượng vanishing gradient.	- output của Multi-Headed Attention (L, 512) - $E_{\{t-1\}}$ (L, 512)	(L, 512) (tổng theo từng phần tử)	Không có tham số
Layer Norm.	Layer Normalization trên output của skip connection. Đưa dữ liệu về phân phối chuẩn (mean 0, var 1) rồi nhân với hệ số tỉ lệ (γ) và cộng bias (β).	(L, 512)	(L, 512)	- γ (gain): (512,) - β (shift): (512,) Cả hai đều là tham số học được

Feed Forward	Mạng truyền thẳng (FFN) gồm 2 lớp tuyến tính với ReLU ở giữa. Áp dụng độc lập từng vị trí.	(L, 512)	(L, 512)	- W1 (512, 2048), b1 (2048) - W2 (2048, 512), b2 (512)
$\oplus$ (Skip connection)	Cộng đầu vào của Feed Forward (Output của Layer Norm bước 3) với đầu ra của Feed Forward.	- output của Feed Forward (L,512) - output của Layer Norm bước 3 (L,512)	(L, 512)	Không có tham số
Layer Norm.	Layer Normalization lần cuối, chuẩn hóa kết quả sau skip connection. Output chính là E_t – đầu ra của encoder layer này, Sau đó làm input cho encoder layer tiếp theo (hoặc đi đến decoder nếu là tầng cuối cùng).	(L, 512)	(L, 512)	- γ (512,), β (512,) (các tham số khác với Layer Norm ở bước 3)

Lưu ý:

- + E_{t-1} là output của encoder layer trước đó (với tầng đầu tiên, E_0 chính là embedding + positional encoding).
- + E_t là output của encoder layer hiện tại, có cùng kích thước (L, 512).
- + 6 lớp trong Encoder không chia sẻ tham số/trọng số (weights) với nhau. Chúng có cấu trúc giống hệt nhau, nhưng các ma trận trọng số trong mỗi lớp là hoàn toàn độc lập và được học riêng biệt.
- + Xuyên suốt tất cả các lớp, kể cả embedding layer, kích thước vector luôn cố định ở $d_{\text{model}} = 512$ để Residual Connection có thể hoạt động.
- + Tính độc lập của FFN: Khối Feed-Forward áp dụng song song và riêng rẽ cho từng token. FFN không "kết nối" các từ với nhau.
- + Cơ chế Attention trong Encoder là Không che (Unmasked): Từ đứng trước hoàn toàn được "chú ý" tới các từ đứng sau để hiểu trọn vẹn ngữ cảnh ngữ nghĩa

Ví dụ: Input: "Tôi đang training **mạng nơ-ron** bằng một **tập dữ liệu** mới để đánh giá xem **nó** có tốt với **dữ liệu này** ko?".

- Từ "**mạng nơ-ron**" biến thành một dãy 512 số: [0.85, -0.12, 0.43, ..., 0.91]
- Từ "**nó**" biến thành một dãy 512 số: [0.10, 0.02, -0.05, ..., 0.11] (*Lúc này vector của từ "nó" chỉ mang ý nghĩa chung chung của một đại từ*)

Sau đó, mô hình cộng thêm **Positional Encoding** vào các vector này để giữ thứ tự trước sau. Sẽ thu được ma trận đầu vào E_0 đi vào tầng Encoder đầu tiên.

Kết quả tính toán (Attention Score) sẽ trả về tỷ lệ phần trăm "sự chú ý" của từ "**nó**" phân bổ cho các từ trong câu:

- Điểm chú ý đối với chính nó: 5%
- Điểm chú ý đối với từ "**tập dữ liệu**": 1%
- Điểm chú ý đối với từ "**mạng nơ-ron**": **85%**

Khi ra khỏi lớp Encoder 6, tất cả các từ vẫn giữ nguyên kích thước vector ban đầu (512 chiều), nhưng giá trị các con số bên trong đã thay đổi hoàn toàn:

- Vector ban đầu của từ "**nó**": [0.10, 0.02, -0.05, ..., 0.11] (Không có thông tin).
- Vector đầu ra cuối cùng của từ "**nó**": [0.82, -0.45, 0.73, ..., 0.88] (Đã biến đổi mang cấu trúc năng lượng ngữ nghĩa trùng khớp với "**mạng nơ-ron**").

Ma trận context này được chuyển sang cho Decoder để dịch một cách chính xác.

PHẦN 3 Huấn Luyện và Kết Quả

Phần 3.2 Kết quả thực nghiệm & Ablation Studies

3.2.1. Mục Tiêu

Báo cáo này phân tích phần đánh giá thực nghiệm (Results) và nghiên cứu biến thể mô hình (Ablation Study) trong bài báo "Attention Is All You Need" (Vaswani et al., 2017).

Sau khi đọc báo cáo, người đọc có khả năng:

- Nắm bằng chứng định lượng về hiệu năng của Transformer so với các mô hình trước.
- Phân tích vai trò của từng siêu tham số (hyperparameter) dựa trên số liệu Ablation Study.

3.2.2. Ý Tưởng Chính

Mục tiêu thực nghiệm là chứng minh định lượng rằng: thay thế toàn bộ cơ chế tuần hoàn (RNN) và tích chập (CNN) bằng Self-Attention thuần túy giúp mô hình đạt độ chính xác cao hơn với chi phí tính toán thấp hơn.

Bốn định hướng thực nghiệm được thiết kế:

- **Đa quy mô dữ liệu:** Kiểm tra trên tập Anh-Đức (4,5 triệu cặp câu) và tập Anh-Pháp (36 triệu câu).
- **Scaling (co giãn cấu trúc):** Xây dựng Transformer-base (tối ưu chi phí) và Transformer-big (tối đa hiệu năng).
- **Ablation Study (bóc tách thành phần):** Thay đổi từng siêu tham số độc lập để xác định đóng góp của từng thành phần kiến trúc.
- **Cross-domain Generalization:** Áp dụng mô hình dịch thuật cho bài toán phân tích cú pháp để kiểm tra tính đa năng.

3.2.3. Kết Quả Thực Nghiệm

1. Tổng Quan Dữ Liệu & Mô Hình

Bộ dữ liệu: Hai bộ dữ liệu chuẩn WMT 2014 được sử dụng. Bộ Anh-Đức gồm 4,5 triệu cặp câu, mã hóa theo Byte-Pair Encoding (BPE) với từ vựng ~37.000 token. Bộ Anh-Pháp lớn hơn 8 lần (36 triệu câu), mã hóa theo Word-Piece với ~32.000 token. BPE và Word-Piece đều là phương pháp mã hóa subword, giúp xử lý từ hiếm và rút ngắn độ dài chuỗi.

Mô hình: Transformer-base ($d_{\text{model}} = 512$, $h = 8$, $d_{\text{ff}} = 2048$, 65 triệu tham số) phục vụ tối ưu chi phí. Transformer-big ($d_{\text{model}} = 1024$, $h = 16$, $d_{\text{ff}} = 4096$, 213 triệu tham số) hướng tới tối đa hiệu năng (SOTA).

Thang đo: BLEU đo chất lượng dịch (trùng khớp n-gram); FLOPs đo chi phí tính toán; PPL (Perplexity) đo độ bất định của mô hình; F1 đánh giá bài toán phân tích cú pháp.

2. Kết Quả Dịch Máy

Hai nhiệm vụ được đánh giá trên WMT 2014: Anh-Đức (EN-DE) và Anh-Pháp (EN-FR). Kết quả chi tiết theo Bảng 2 gốc của bài báo:

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Transformer-big đạt 28.4 BLEU trên EN-DE, vượt hệ thống gộp tốt nhất (ConvS2S Ensemble: 26.36 BLEU) trong khi FLOPs thấp hơn hơn 3 lần. Trên EN-FR, Transformer-big đạt 41.8 BLEU — vượt mọi mô hình đơn lẻ trước đó với chi phí huấn luyện chỉ bằng 1/50 hệ thống gộp tốt nhất.

Ý nghĩa: Self-Attention không làm giảm độ chính xác; loại bỏ tính tuần tự không mất khả năng nắm bắt ngữ cảnh. Song song hóa toàn diện giúp rút ngắn thời gian huấn luyện từ nhiều tuần xuống còn tính bằng ngày. Hiệu năng tăng nhất quán khi mở rộng quy mô (base → big), tạo tiền đề cho các mô hình ngôn ngữ lớn (LLM) sau này.

3.2.4. Ablation Study – Biến Đổi Siêu Tham Số

Ablation Study thay đổi từng siêu tham số độc lập, giữ nguyên các biến còn lại, đánh giá trên tập phát triển newstest2013 (EN-DE). Kết quả đầy đủ theo Bảng 3 gốc:

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ts}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)				16						5.16	25.1	58
				32						5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)										4.92	25.7	
big	6	1024	4096	16			0.3		300K	4.33	26.4	213

development set: newstest2013. We used beam search as described in the previous section, but no

Nhóm (A) – Số đầu chú ý (h) và kích thước vector (d_k, d_v)

Giữ nguyên tích $h \times d_k = 512$. Khi $h = 1$ (đầu đơn lẻ), điểm giảm xuống 24.9 BLEU.

Khi $h = 32$ ($d_k = 16$), điểm cũng giảm xuống 25.4 so với 25.8 của cấu hình chuẩn $h = 8$. Cơ chế đa đầu cần thiết để học đồng thời nhiều loại liên kết ngữ nghĩa, nhưng kích thước mỗi đầu không được quá nhỏ.

Nhóm (B) – Kích thước khóa chú ý (d_k)

Giảm d_k độc lập xuống 16 và 32 làm BLEU giảm tương ứng xuống 25.1 và 25.4. d_k quyết định độ chính xác của phép đo tương đồng ngữ cảnh; giá trị quá nhỏ hạn chế chất lượng dot-product attention.

Nhóm (C) – Quy mô mô hình

Tăng N , d_{model} hoặc d_{ff} đều cải thiện điểm số nhất quán. Cấu hình $d_{\text{ff}} = 4096$ đạt 26.2 BLEU so với 25.8 của cơ sở. Transformer có khả năng mở rộng quy mô tốt — mô hình lớn hơn cho hiệu năng cao hơn theo hướng có thể dự đoán.

Nhóm (D) – Dropout và Label Smoothing

Tắt Dropout ($P = 0.0$) làm BLEU giảm xuống 24.6. Tắt Label Smoothing ($\epsilon = 0.0$) cải thiện PPL (4.67) nhưng làm BLEU thực tế giảm. Transformer có xu hướng overfitting; Dropout và Label Smoothing tuy tăng PPL khi huấn luyện nhưng cải thiện khả năng tổng quát hóa.

Nhóm (E) – Phương pháp nhúng vị trí (Positional Encoding)

Thay hàm lượng giác (sinusoidal) bằng vector vị trí học được cho kết quả giống nhau: 25.7 BLEU và 4.92 PPL. Hàm lượng giác được ưu tiên vì cho phép mô hình ngoại suy tốt hơn với câu dài hơn phạm vi huấn luyện.

Cấu hình Transformer-big được tổng hợp từ kết quả nhóm A–E: $h = 16$, $d_{\text{model}} = 1024$, $d_{\text{ff}} = 4096$, 213 triệu tham số, 300.000 bước huấn luyện. Kết quả: 4.33 PPL và 26.4 BLEU trên tập phát triển — mức tốt nhất trong toàn bộ bảng.

3.2.5. Thử Nghiệm Tính Đa Năng – Phân Tích Cú Pháp

Để kiểm tra khả năng tổng quát hóa, Transformer (4 tầng, $d_{\text{model}} = 1024$) được áp dụng vào bài toán English Constituency Parsing — chuyển câu văn sang cây cú pháp cấu trúc. Đây là bài toán khác biệt hoàn toàn với dịch thuật: đầu ra chịu ràng buộc cấu trúc nghiêm ngặt và thường dài hơn đầu vào. Siêu tham số giữ nguyên từ mô hình dịch Anh-Đức, không tinh chỉnh theo đặc thù tác vụ.

Dữ liệu kiểm thử: Section 23 của Penn Treebank (WSJ). Thang đo: điểm F1. Hai chế độ: WSJ only (~40.000 câu) và Semi-supervised (~17 triệu câu).

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

Ở chế độ WSJ only, Transformer đạt 91.3 F1 — vượt hầu hết mô hình RNN/CNN chuyên biệt, chỉ đứng sau Dyer et al. (91.7). Ở chế độ bán giám sát, Transformer đạt 92.7 F1 — cao nhất trong nhóm so sánh tương đồng (discriminative).

Lưu ý: Luong et al. (93.0) và Dyer et al. (93.3) dùng phương pháp multi-task và generative, khác bản chất với Transformer (discriminative), nên không so sánh trực tiếp.

Kết quả xác nhận rằng Self-Attention không bị giới hạn trong một tác vụ: mô hình có thể tự học cấu trúc ngôn ngữ tổng quát mà không cần tinh chỉnh, kể cả trong điều kiện ít dữ liệu.

3.2.6. Tổng Kết

Bài báo "Attention Is All You Need" cung cấp bằng chứng định lượng qua ba hướng thực nghiệm độc lập:

- **Dịch máy (Bảng 2):** Transformer-big đạt SOTA 28.4 BLEU (EN-DE) và 41.8 BLEU (EN-FR), vượt mọi mô hình đơn lẻ và hệ thống gộp trước đó với chi phí FLOPs thấp hơn nhiều lần.
- **Ablation Study (Bảng 3):** $h = 8$, $d_k = 64$, Dropout = 0.1, Label Smoothing = 0.1 là cấu hình tối ưu. Mô hình lớn hơn luôn cho kết quả tốt hơn. Regularization bắt buộc để tránh overfitting.
- **Phân tích cú pháp (Bảng 4):** 92.7 F1 trên constituency parsing mà không tính chỉnh siêu tham số, khẳng định khả năng tổng quát hóa đa tác vụ của Self-Attention.

Kiến trúc Transformer dựa hoàn toàn trên Self-Attention vừa đạt hiệu năng cao hơn, vừa tiêu tốn ít tài nguyên hơn so với RNN và CNN — và tạo nền tảng cho toàn bộ thế hệ mô hình ngôn ngữ lớn (LLM) phát triển sau này.

PHẦN 4 —Phân tích & Tác động

Ưu điểm/Nhược điểm của Transformer[An Phú]

1.Ưu điểm của Transformer:

-Thứ nhất nó hỗ trợ song song hóa tốt vì không cần xử lý tuần tự từng bước như RNN, nên tận dụng phần cứng hiệu quả hơn và thường huấn luyện nhanh hơn trên GPU
Vì sao nó lại có khả năng đó? Bởi vì ban đầu RNN đọc từng token theo thứ tự, muốn tính token sau phải chờ token trước, nó phụ thuộc tuần tự. Trong khi đó Transformer thì đưa cả chuỗi vào cùng lúc và xử lý bằng các phép biến đổi trên ma trận, nên các token sau không còn phải chờ token trước đó nữa.→Cũng chính điều này giúp cho GPU làm việc hiệu quả hơn bởi GPU mạnh ở các phép tính ma trận lớn và song song

→Nói một cách dễ hiểu là thay vì “đọc từng chữ một” như RNN thì Transformer “nhìn cả câu cùng lúc”

-Thứ hai Transformer có độ dài đường đi là $O(1)$ giữa các token trong self-attention nghĩa là thông tin từ một vị trí có thể đến vị trí khác trực tiếp một lớp attention mà không phải đi qua nhiều bước trung gian

Vì sao có đường đi $O(1)$? Vì trong self-attention mỗi token có thể tính mức liên quan với mọi token khác ngay trong cùng một lớp

→ Trong self-attention, mỗi token giống như có một 'đường hầm' nối thẳng đến các token khác. Token nào có trọng số liên quan cao thì đường hầm đó mạnh hơn, nên thông tin đi qua dễ hơn; token nào liên quan thấp thì đường hầm yếu hơn

-Thứ ba multi-head attention cho phép mô hình nhìn dữ liệu từ nhiều "góc nhìn" cùng lúc thay vì chỉ học một kiểu quan hệ duy nhất (được nói rất chi tiết ở phần của Nhật Anh)

Tại sao cần nhiều kiểu quan hệ cùng lúc? Bởi nếu muốn xử lý được ngôn ngữ tự nhiên không thể nào chỉ với một 1 head x 512 chiều bởi trong 1 câu đã có quan hệ cú pháp, ngữ nghĩa,.....nên ta cần chia ra vừa đủ để quản lí (quá nhiều cũng dễ bị trùng lặp chức năng)

→ Ưu điểm này ta có thể hiểu là Transformer không chỉ biết "ai liên quan đến ai" mà còn biết được thêm nhiều dạng liên quan khác nữa làm cho khả năng biểu đạt tốt hơn

-Thứ tư attention visualization làm cho Transformer có phần dễ diễn giải hơn vì ta có thể quan sát mức độ chú ý giữa các token, từ đó phần nào hiểu được vì sao mô hình đưa ra dự đoán

Tại sao ta có thể thấy được hay quan sát được mức độ này? Là bởi vì ta hoàn toàn có công thức được show ra ở mục Scaled Dot với hàm softmax thì ta hoàn toàn có thể trực quan hóa dữ liệu bằng các loại đồ thị

→ Nói tóm lại thì Transformer cho phép ta quan sát một phần mối quan hệ giữa các token vì attention weights là đầu ra trung gian được tính rõ ràng trong cơ chế attention, nên có thể trích xuất và trực quan hóa.

-Cuối cùng thì Transformer có khả năng mở rộng tốt, nên khi tăng dữ liệu, số tham số và tài nguyên tính toán, mô hình thường tiếp tục cải thiện mạnh → Đây cũng chính là lý do mà mô hình này trở thành nền tảng của nhiều mô hình ngôn ngữ và AI hiện đại

2.Nhược điểm của Transformer:

-Tốn GPU/VRAM bởi self-attention phải xét quan hệ giữa mọi token, nên chi phí tăng nhanh khi câu dài

-Cần dữ liệu lớn: Transformer thường phát huy tốt khi có nhiều dữ liệu; nếu dữ liệu ít, mô hình dễ học kém hiệu quả hơn.

-Huấn luyện đắt đỏ: Thời gian và chi phí huấn luyện thường cao do mô hình lớn và nhiều phép tính.

-Khó với chuỗi rất dài: Khi độ dài đầu vào tăng, bộ nhớ và độ phức tạp tính toán cũng tăng đáng kể.

-Yêu cầu tinh chỉnh cẩn thận: Với số lượng tham số lớn, việc chọn hyperparameter, regularization và dữ liệu phù hợp quan trọng hơn.

Và có một vấn đề về nhược điểm của LLM và Transformer mà mọi ng sẽ thấy thông qua bài báo <https://tiasang.com.vn/nha-khoa-hoc-che-benh-gia-ai-lai-khang-dinh-that-5061375.html> Sơ qua về bài báo này thì một chuyên gia về y tế đã đăng tải hai bài báo lên một trang lưu trữ các bài báo chưa được bình duyệt và chỉ sau vài tuần khi chuyên gia đó hỏi LLM thì nó đã cập nhật căn bệnh giả này. Vậy đứng từ góc độ người hiểu về LLM hiểu về Transformer thì ta sẽ cùng tìm hiểu nhược điểm mà LLM mắc phải ở đây là gì?

[Self-Attention] cho phép mô hình gán trọng số cho các token trong câu theo mức liên quan với nhau. Khi một thuật ngữ giả được viết giống thuật ngữ thật, các token trong cụm đó có thể nhận trọng số cao chỉ vì chúng “trông hợp ngữ cảnh” chứ không phải chúng đúng

[Q-K-V] Trong attention, token tạo ra query, còn các token khác cung cấp key/value để mô hình so khớp và lấy thông tin. Nếu đầu vào chứa một cụm từ giả nhưng có hình thức gần giống tài liệu khoa học, query có thể khớp mạnh với các pattern quen thuộc trong dữ liệu huấn luyện, khiến mô hình ưu tiên nó

[Positional encoding] Transformer cần positional encoding để biết thứ tự token, nhưng cơ chế này chỉ giúp hiểu cấu trúc chuỗi, không giúp biết câu đó có đúng ngoài thực tế hay không. Nó cho mô hình biết “cái gì đứng cạnh cái gì”, chứ không cho mô hình biết “cái gì là sự thật”.

[Huấn luyện theo mẫu ngôn ngữ] Transformer học từ rất nhiều văn bản nên nó mạnh ở việc bắt mẫu xuất hiện của ngôn ngữ. Nhưng nếu một thông tin giả có văn phong rất giống tài liệu thật, mô hình có thể coi đó là pattern hợp lệ và “bịa” tiếp theo hướng đó.

→ Transformer và các LLM rất mạnh trong việc hiểu và sinh ngôn ngữ, nhưng chúng không có cơ chế tự kiểm chứng sự thật độc lập. Vì thế, nếu đầu vào hoặc dữ liệu huấn luyện có sai lệch, mô hình vẫn có thể tạo ra câu trả lời sai. Điều này cho thấy con người vẫn cần học để biết cách đánh giá, lọc và kiểm chứng thông tin, thay vì phụ thuộc hoàn toàn vào AI.